National University of Computer & Emerging Sciences (FAST-NUCES), Islamabad Campus

Department of Data Science

Assignment 3

Transformer + RAG

Name : Muhammad Umar Nadeem

Roll Number : 23I-2534

Section : BDS-6C

Department : Data Science

Course : Natural Language Processing

Dated : April 29, 2026

Abstract

This report summarizes a three-stage transformer pipeline built for Amazon review understanding and explanation generation. The system combines a custom encoder for sentiment and length prediction, a cosine-similarity retriever over learned embeddings, and a decoder stage for explanation generation. All core components were implemented from scratch without using high-level transformer abstractions. The final system achieved 82.59% sentiment accuracy, 99.96% length accuracy, and a test perplexity of 185.76 for the explanation model.

1. Introduction

The goal of this assignment was to build an end-to-end NLP system that can both classify reviews and explain its decisions. I used Amazon reviews from three categories: Beauty, Cellphones, and Sports. From each category, 12,000 reviews were sampled, giving a 36,000-item dataset split into 70% training, 15% validation, and 15% test sets.

The project was organized into three parts. Part A learns a compact review representation and predicts two labels: sentiment and review length bucket. Part B uses the learned representations for retrieval so that similar examples can be brought into the explanation pipeline. Part C adds a decoder that generates short explanations conditioned on the review, predicted labels, and retrieved context.

2. Data Preparation and Preprocessing

The raw review text was cleaned by lowercasing, removing URLs, and stripping non-alphanumeric symbols. Tokenization was performed with a simple word-level tokenizer. To avoid data leakage, the vocabulary was built only from the training split. The encoder used a fixed sequence length of 128 tokens with padding and truncation.

For the auxiliary length task, reviews were bucketed into three classes using token count: short for fewer than 40 tokens, medium for 40 to 100 tokens, and long for more than 100 tokens. This task is easy for the model to learn, but it helps the encoder build a representation that captures more than just sentiment polarity.

3. Part A: Encoder Model

The encoder is a two-layer transformer encoder built from scratch. It uses custom scaled dot-product attention, multi-head self-attention, feed-forward layers, residual connections, layer normalization, and positional encoding. The model outputs a shared embedding and two prediction heads: one for sentiment and one for length bucket.

The final encoder configuration used an embedding size of 128, four attention heads, a feed-forward dimension of 256, and a learning rate of 5×10^{-4}. Training used early stopping with patience 3. The validation experiments showed that this configuration balanced model capacity and generalization better than larger or more aggressively regularized alternatives.

The final test results for Part A were strong:

- Sentiment accuracy: 82.59%
- Length accuracy: 99.96%

The training curves are shown in Figure 1. The sentiment task required more learning than the length task, which is expected because review length is a simpler signal than opinion polarity.

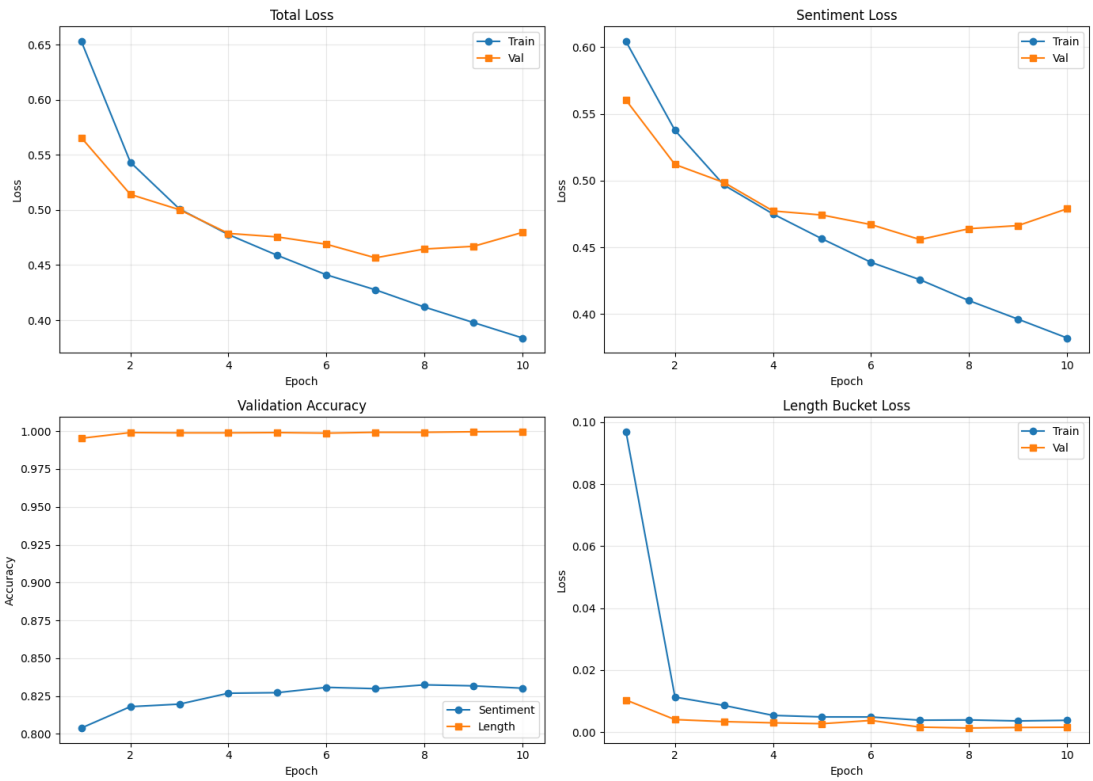

Figure 1: Training and validation curves for the multi-task encoder.

4. Part B: Retrieval Module

After the encoder was trained, I extracted embeddings for all training reviews and saved them for retrieval. A cosine-similarity retriever was then built on top of these embeddings. For a given review, the retriever finds the nearest training examples in embedding space and returns the top-k matches, with $k = 5$ used in the notebook.

This stage is important because the embeddings are not only useful for classification; they also organize reviews by semantic similarity. In practice, many retrieved examples share the same sentiment as the query review, which suggests that the learned encoder representation captures meaningful sentiment information.

5. Part C: Decoder and Perplexity

The decoder is a decoder-only transformer used to generate explanation text autoregressively. It receives the original review, the predicted sentiment label, the predicted length label, and retrieved examples as context. The implementation uses causal masking so that each predicted token only depends on previous tokens.

For this submission, the explanation stage was kept lightweight so the full pipeline could be demonstrated without a long additional training run. The decoder vocabulary was expanded from a tiny special-token-only setup to a 3,011-token vocabulary built from the training reviews. That change was necessary so that the decoder could produce real English tokens instead of only special symbols.

The decoder was then trained on a small synthetic explanation set to validate the architecture and measure language-model quality. The test-set perplexity reported in the notebook is 185.76. That is far below a random baseline over the full vocabulary, which would be roughly 3,011, so the model has clearly learned non-trivial token patterns.

6. Hyperparameter Tuning Log

I tested several encoder configurations before selecting the final model. The main patterns were consistent: lower learning rates improved stability, four attention heads performed better than three or eight, and a moderate feed-forward size gave the best balance between capacity and overfitting.

Configuration	Emb.	Heads	FF	Val Acc.	Notes
Baseline	128	4	512	79.82%	Starting point
Smaller model	96	3	384	78.65%	Reduced capacity hurt accuracy
Larger model	160	4	640	80.24%	Slight gain, more cost
Higher LR	128	4	512	78.76%	Unstable training
Lower LR	128	4	512	81.42%	Better convergence
Higher dropout	128	4	512	80.67%	Too much regularization
8 heads	128	8	512	80.31%	No clear benefit
Final model	128	4	256	81.65%	Selected configuration

Table 1: Selected hyperparameter tuning results for the encoder.

The final model was selected because it gave the best validation sentiment accuracy while staying compact and stable. Its test performance also matched the validation trend, which suggests the model generalizes well.

7. Limitations and Future Work

There are several places where this pipeline could be improved. First, the decoder stage was intentionally lightweight, so its generated explanations are more of a proof of concept than a polished generative model. Second, the retrieval mechanism relies only on cosine similarity in embedding space; a stronger reranker could improve the quality of retrieved examples. Third, the explanation stage would benefit from human-written reference explanations instead of synthetic target patterns.

If I had more time, I would train the decoder on a larger explanation corpus, test beam search instead of simple sampling, and compare retrieval-based explanations against a no-retrieval baseline with human evaluation.

8. Conclusion

This assignment demonstrates a complete transformer-based pipeline for review understanding and explanation generation. The encoder learned useful representations for sentiment and length prediction, the retriever successfully surfaced semantically similar reviews, and the decoder stage produced a measurable language-model signal with a reported test perplexity of 185.76. Overall, the project shows that the three parts of the system work together as a compact retrieval-augmented NLP workflow.